Plant Disease Prediction

Project Progress Report - 2026-10-09

Executive summary

This report summarizes the Plant Disease Prediction project work completed through the advanced model-analysis dashboard and model audit. The application now has a safer, more transparent pipeline from image upload to model inference, canonical disease mapping, routing, uncertainty handling, and user-facing evidence.

The project uses a FastAPI backend, TensorFlow/Keras and ONNX model artifacts, a React/Vite frontend, a versioned canonical disease registry, reproducible audit utilities, and automated regression tests.

What is now more advanced

1. Dataset governance: read-only dataset auditing, corruption checks, cryptographic duplicate detection, label validation, and explicit unavailable-dataset reporting.

2. Canonical disease identity: stable crop-specific IDs, preserved source labels, healthy classes, registry validation, lookup helpers, and unresolved mapping states.

3. Reproducible ML infrastructure: safe split manifests, leakage checks, training guards, benchmark utilities, calibration and abstention infrastructure, and OOD/image-quality checks.

4. Model adapters: a shared adapter contract validates preprocessing, checkpoint shape, output dimensions, score semantics, canonical mappings, and inference failures.

5. Deterministic routing: only verified ready models are eligible; configured priority selects the production model; no incompatible probability averaging or silent fallback occurs.

6. End-to-end interface: upload validation, drag and drop, preview, request cancellation, stale-response protection, model evidence cards, decision trace, and honest uncertainty states.

7. Model audit and dashboard: the system now evaluates both verified EfficientNet artifacts independently and displays unavailable models with their actual blockers.

Current verified model status

ONNX EfficientNet-B0 (plantvillage-efficientnetb0-onnx): READY. The 224x224 RGB input, 38-class output, normalized probability output, source-label order, and canonical mappings were verified. It remains the higher-priority production route.

Keras EfficientNet-B0 (plantvillage-efficientnetb0-keras): READY. The local Keras checkpoint loads successfully with shape (None, 224, 224, 3) to (None, 38). Its indexed class_names.txt matches the canonical PlantVillage mappings and real inference succeeds.

MobileNetV2 PyTorch checkpoint (mobilenetv2-plant-pth): UNAVAILABLE. PyTorch is not installed in the environment, and the architecture, preprocessing, output mapping, and class metadata are not sufficiently verified. The checkpoint is not treated as production-ready.

Plant Disease Prediction

Project Progress Report - continued

ONNX prediction validation

The ONNX output is validated as a finite, non-negative, normalized probability distribution. The class-index regression test confirms index 16 maps to Peach___Bacterial_spot, canonical ID peach.bacterial_spot, and display label Bacterial spot.

The score is explicitly uncalibrated. It must not be presented as a validated probability that the biological diagnosis is correct. The reported model inference duration measures the model execution section rather than the full network request.

Calibration remains unavailable because the existing calibration artifact is logits-based while the verified ONNX model emits probabilities. No new threshold was invented.

Backend and API improvements

POST /predict remains backward-compatible. POST /predict/router exposes the Step 9 structured routing result. POST /analyze now returns request ID, image validation, per-model analysis, score type, top predictions, canonical mappings, latency, warnings, exclusions, final decision, and benchmark metadata.

Per-model analysis is isolated: an unavailable or failed model does not fabricate a result and does not prevent other verified models from being reported.

The repeatable diagnostic command is scripts/diagnose_models.py. It writes reports/model_diagnostics.json and can optionally run a real image probe without modifying model artifacts.

Frontend dashboard improvements

The React dashboard provides image upload and drag-and-drop, client-side type and 10 MB validation, preview and reset, duplicate-request prevention, request cancellation, and stale-result protection.

The results area distinguishes successful, uncertain, abstained, unavailable, and warning states. It displays canonical labels, healthy status, score type, calibration status, latency, limitations, and actual model exclusions.

The analysis dashboard includes individual model cards, top-prediction lists, a current-image score comparison, a router decision trace, and a separate offline benchmark section. Missing benchmark metrics are shown as Not available.

No Grad-CAM or fabricated heatmap was added because a validated explainability pipeline is not available for the current adapter architecture.

Plant Disease Prediction

Project Progress Report - continued

Safety and reliability guarantees

Unknown or unresolved source labels are not silently remapped. Unavailable models are not loaded merely to fill charts. Unsupported crops return an explicit unavailable result.

Raw scores, probabilities, calibrated probabilities, and offline benchmark metrics are kept semantically separate. The system does not claim consensus, voting, score averaging, or biological certainty.

Uploaded images are validated by the backend, are not permanently stored by the prediction endpoint, and server filesystem paths and stack traces are not exposed to users.

Verification completed

Focused adapter, router, and integration tests: 21 tests passed.

Full project regression suite: 51 tests passed.

The model diagnostic command successfully initialized and probed the ONNX and Keras checkpoints. The MobileNet result is recorded as unavailable with explicit reasons.

Python compilation and workspace diagnostics reported no errors for the changed backend and diagnostic modules.

Frontend source diagnostics reported no errors. Vite/Oxlint execution remains pending because Node.js/npm is not available in the current environment.

Remaining limitations and recommendations

PlantVillage and PlantDoc image datasets are not locally available, so real dataset counts and label-distribution benchmarks remain unavailable.

The verified test manifest is empty; accuracy, macro-F1, weighted-F1, recall, calibration, and validated abstention thresholds cannot yet be claimed.

Install Node.js to run the frontend production build and lint commands.

To validate MobileNetV2, install a compatible PyTorch runtime and obtain authoritative architecture, preprocessing, output-label, dataset, and registry metadata.

Next ML step: create approved, duplicate-safe train/validation/test manifests, then generate held-out benchmark and calibration artifacts before interpreting model confidence.